Ember & Bean

Comprehensive Low-Level Technical Design Documentation

Project: B2B coffee Platform
Date: 14 April 2026

1. Architectural Overview & Design Tenets

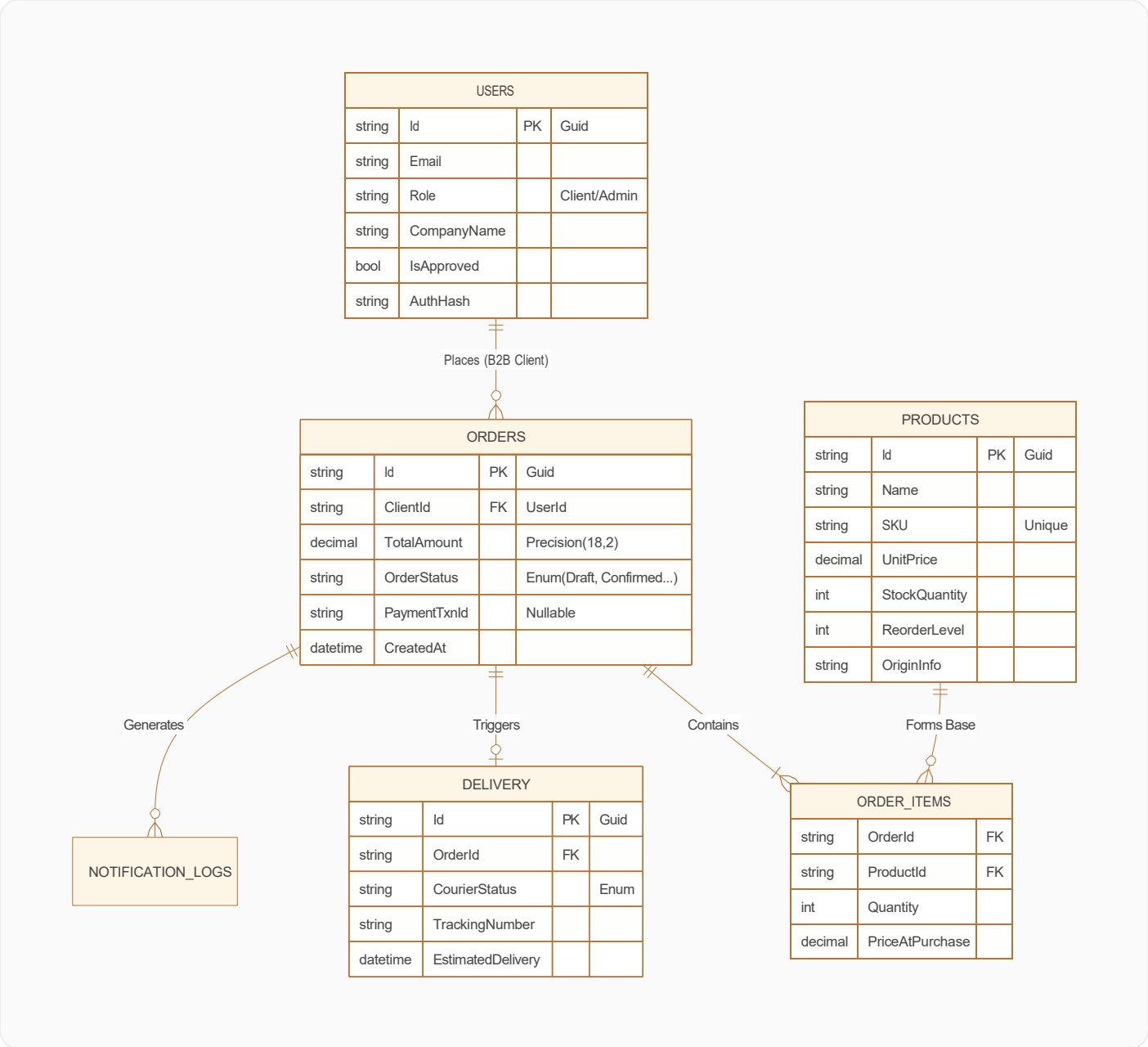
The **Ember & Bean system** is fundamentally structured as a loosely coupled microservices mesh, geared specifically towards high-volume B2B (Business-to-Business) interactions. The platform is engineered to address the complexities of customized bulk pricing, credit vetting, sophisticated delivery routing, and rigorous inventory consistency.

Core Technical Tenets

- **Decentralized Data Management:** Each microservice possesses its own exclusive database context (Database-per-service pattern). This guarantees autonomy and eliminates shared-database bottlenecks.
- **Eventual Consistency via Message Brokerage:** Cross-service synchronization relies on asynchronous domain events published to RabbitMQ exchanges, protecting the critical order completion path from downstream delays or outages.
- **Domain-Driven Design (DDD):** Codebases within core services feature rich domain models encapsulating business rules, rather than relying on anaemic property bags alongside massive procedural logic.
- **Frontend Modularity:** The Angular SPA operates with distinct module boundaries (e.g., Client Portal vs. SuperAdmin Dashboard) supported by standalone components, lazy loading, and granular state management.

2. ER DIAGRAM

Although databases are logically isolated across microservices, this unified Entity-Relationship Diagram maps the cohesive distributed data topography. It visualizes the relational interlock without compromising the physical decoupling enforced by the architecture.

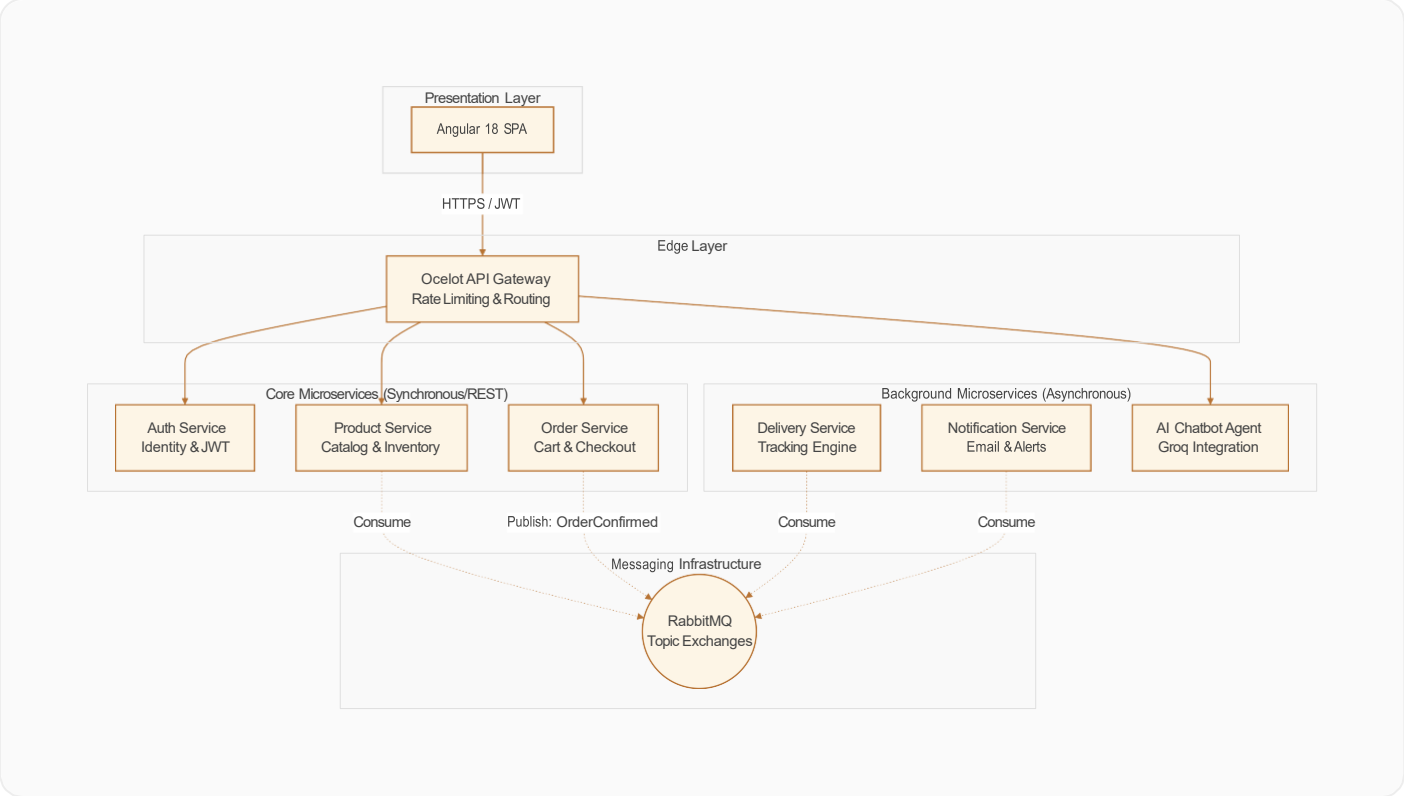


Data Dictionary Focus: Conflict Avoidance

Database interactions prominently apply Optimistic Concurrency Control (OCC) using RowVersion/ConcurrencyTokens within Entity Framework Core. Particularly in `OrderService` and `ProductService`, rapid sequential status updates from administrators or payment callbacks are protected from race conditions.

3. COMPONENT DIAGRAM

This macro-level architectural blueprint details the internal subsystem demarcations. Notice the clear boundary between synchronous API calls (Gateway -> Service) and asynchronous internal invocations (Service -> Message Bus -> Service).

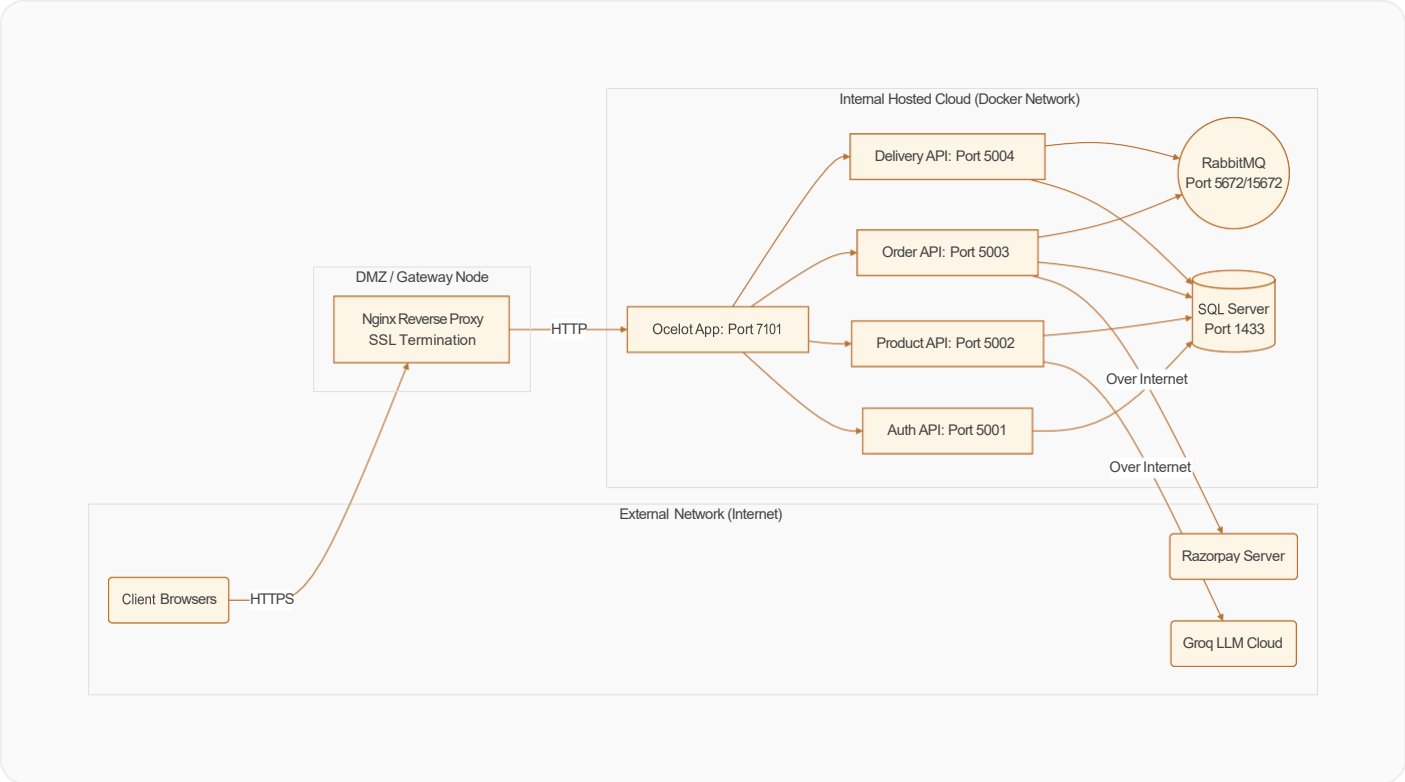


Service Role Definitions

- **Ocelot Gateway:** The sole point of ingress. Maps external `/api *` routes to internal service addresses, trims headers, and intercepts brute-force behaviors.
- **Product Service:** Manages coffee roasts, SKUs, and stock limits. Critical to preventing overselling. Exposes endpoints to adjust stock levels deterministically.
- **Order Service:** Orchestrates the "pending-to-paid" lifecycle. Integrates with Razorpay for synchronous payment validation against transaction signatures.

4. DEPLOYEMENT DIAGRAM

Outlining the physical deployment paradigm, leveraging Linux-based Docker containerization for maximum parity between staging and production tiers.

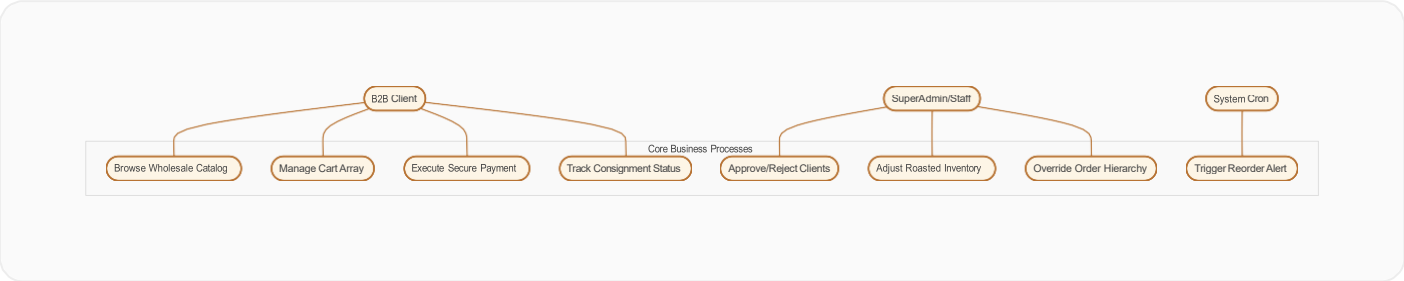


Deployment Logistics

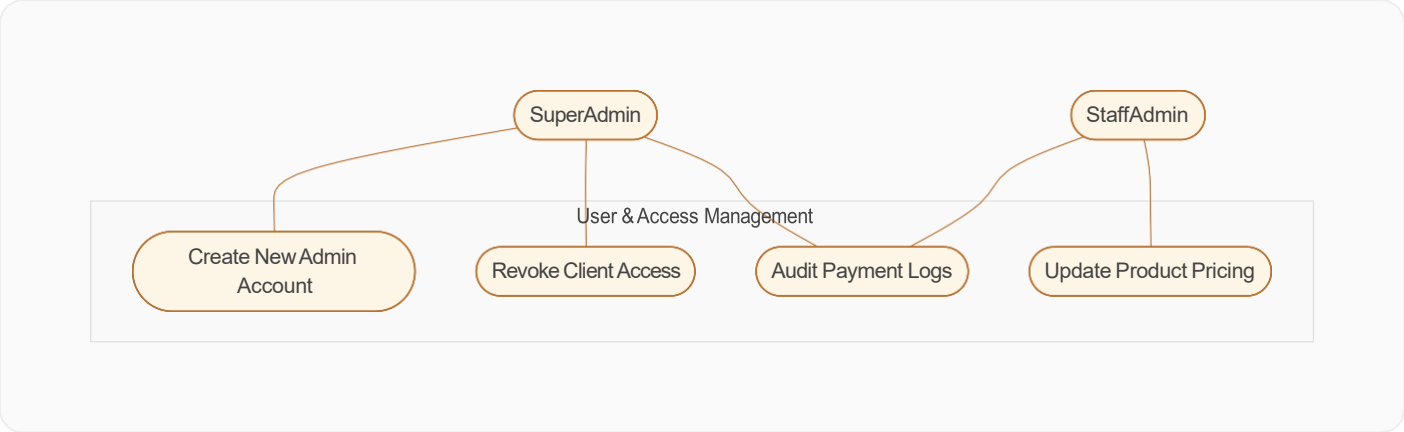
Each service contains its standalone `Dockerfile` based on the multi-stage `mcr.microsoft.com/dotnet/aspnet:8.0` runtime. Local deployment pivots via `docker-compose.yml` mapping respective volumes and ensuring network connectivity to the shared SQL and MQ instances.

5. USE GASE DIAGRAM

Capturing the behavioral interactions from the viewpoint of primary system actors. B2B implies restrictive access schemas, reflected herein.



Administrative Management Sub-Flow



Critical Use Case: B2B Client Onboarding (UC5)

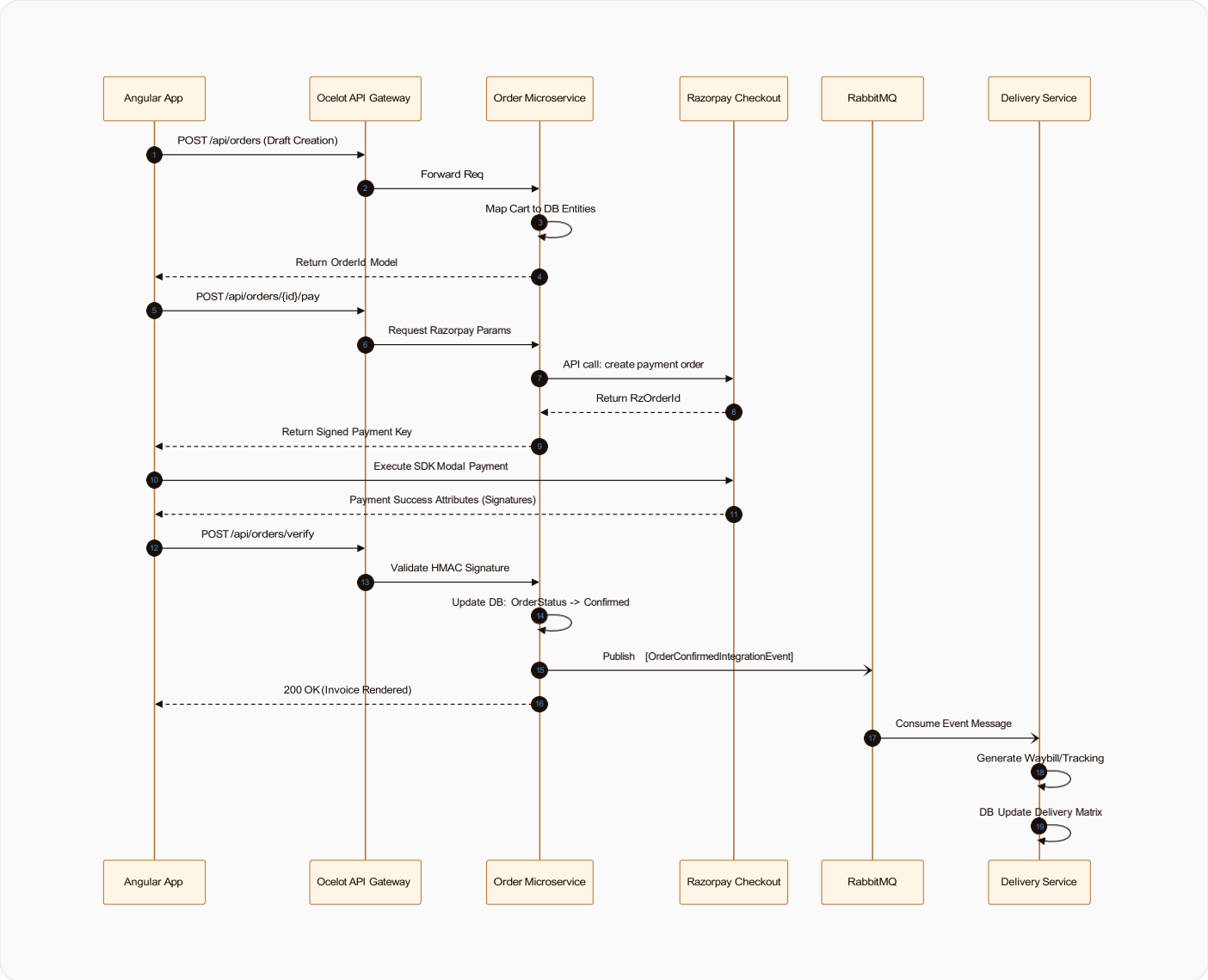
Pre-condition: User registers with tax identification and company profile.

Main Success Scenario: The user attempts login but is met with "Awaiting Authorization". SuperAdmin reviews portal data. Admin flags user as `IsApproved = true` . The user is now capable of receiving the JWT payload permitting catalog reads.

Exception: Disqualification due to inadequate credit credentials results in account suspension; Auth service rejects subsequent token generation attempts.

5. SEQUENCE DIAGRAM

Illustrating the synchronous and asynchronous ballet accompanying successful transaction closure. This sequence specifically tracks the validation of Razorpay callbacks and the subsequent domain event dispatch.

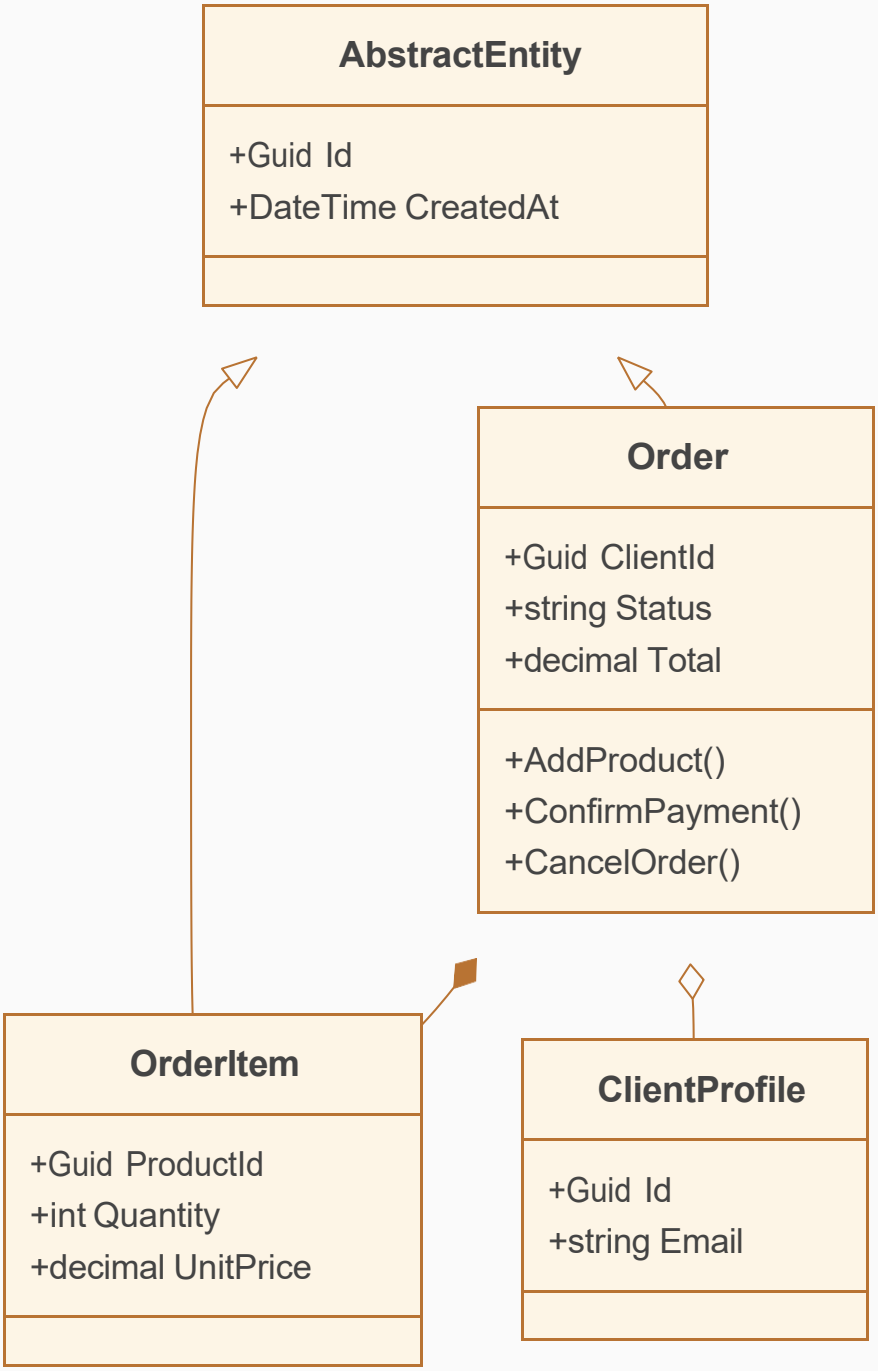


Signature Verification Details

Step 9 verifies authenticity utilizing the HMAC SHA256 cryptographic hashing algorithm. The `OrderService` combines `razorpay_order_id` and `razorpay_payment_id` alongside the localized API secret. Unless the computed hash perfectly matches `razorpay_signature` , the workflow instantly aborts to defend against payload tampering.

7. CLASS DIAGRAM

Focusing deeply on the `OrderService.Domain` structure. We implement pure Domain-Driven Design (DDD). Models safeguard their own invariants; public setters are discouraged in favor of explicit behavior-driven methods.



Domain Invariants

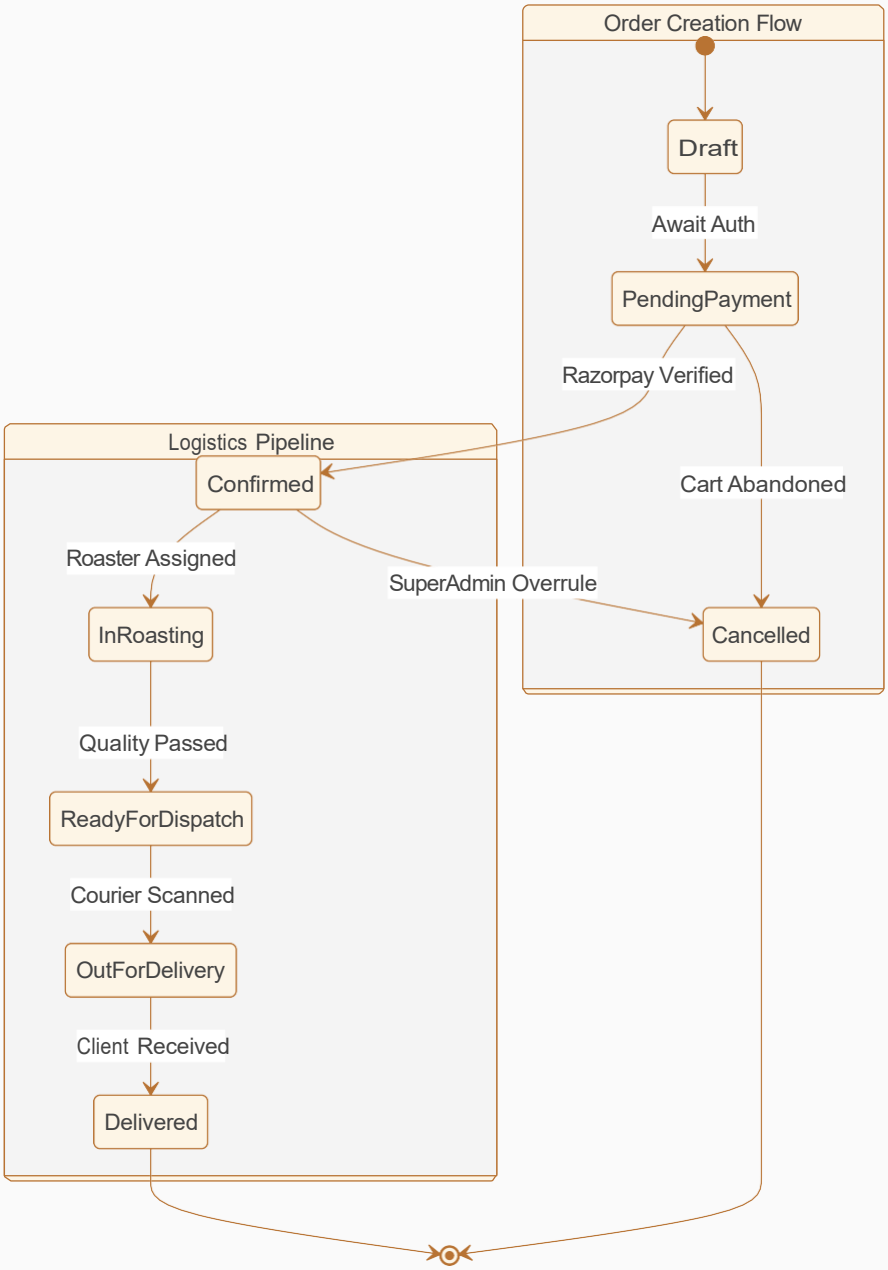
- `AddProduct` guarantees quantity > 0 and validates duplicate product IDs by coalescing quantities.
- `ConfirmPayment` enforces the state machine rule: A payment transaction cannot be attached unless current status is *PendingPayment*.
- The Aggregate Root calculates its own total internal state, assuring external DTOs cannot artificially inject malformed totals.
-

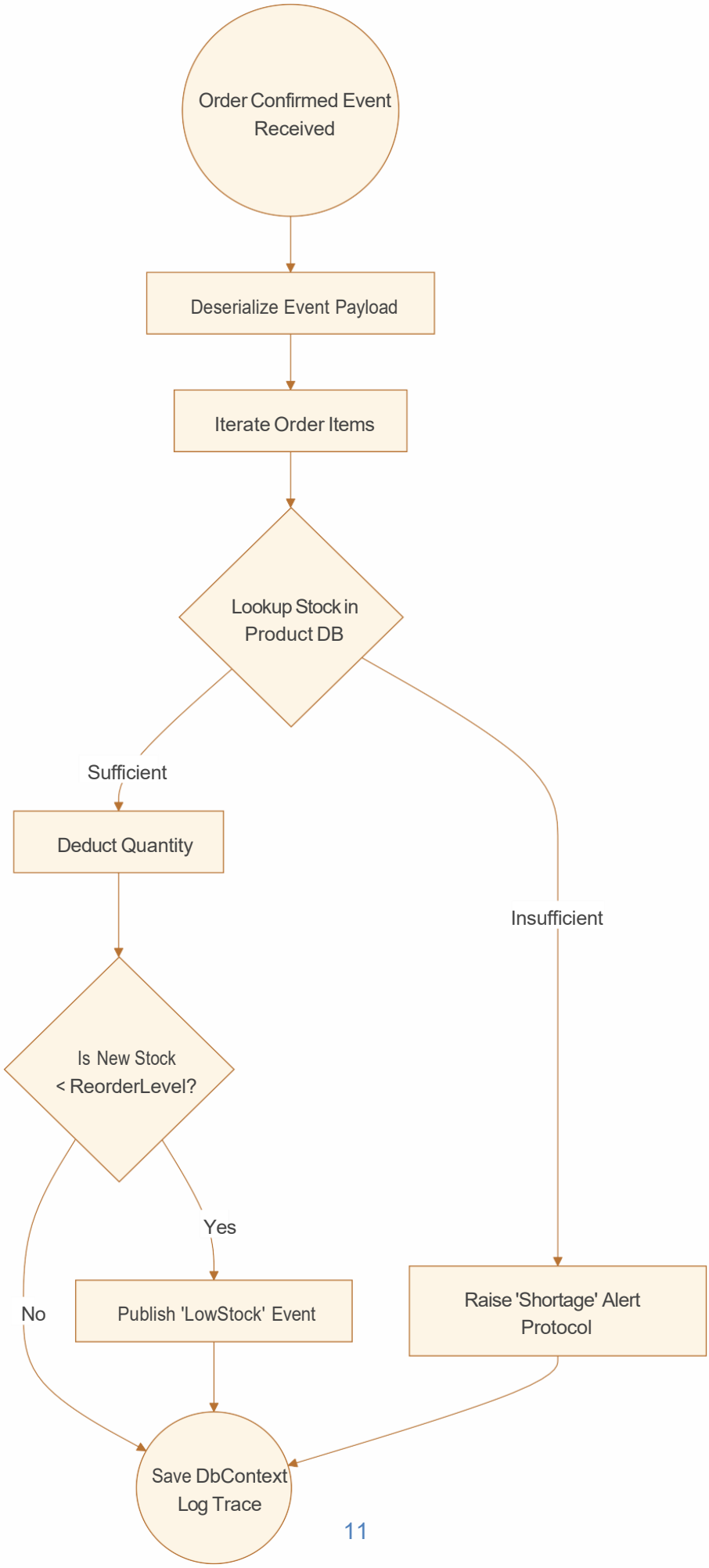
8. STATE DIAGRAM

An exact map of the State Machine governing an Order and its associative Delivery phase. Constraints dictate that back-tracking (e.g., Confirmed going back to Pending Payment) throws an `IllegalStateException` .

Status Mapping Tables

Service Phase	Enum Value	Action Allowed	Trigger Origin
Order	`Confirmed`	Move to InRoasting, Send Invoice Email	Razorpay Webhook / SDK Verification
Delivery	`InRoasting`	Update Tracking UI	SuperAdmin Manual Selection
Delivery	`Delivered`	Archive order, Free up buffer capacity	Delivery Staff/ Integrated Courier


Order Creation Flow
Draft
Await Auth
PendingPayment
Razorpay Verified
Cart Abandoned
Cancelled
Logistics Pipeline
Confirmed
Roaster Assigned
SuperAdmin Overrule
InRoasting
Quality Passed
ReadyForDispatch
Courier Scanned
OutForDelivery
Client Received
Delivered

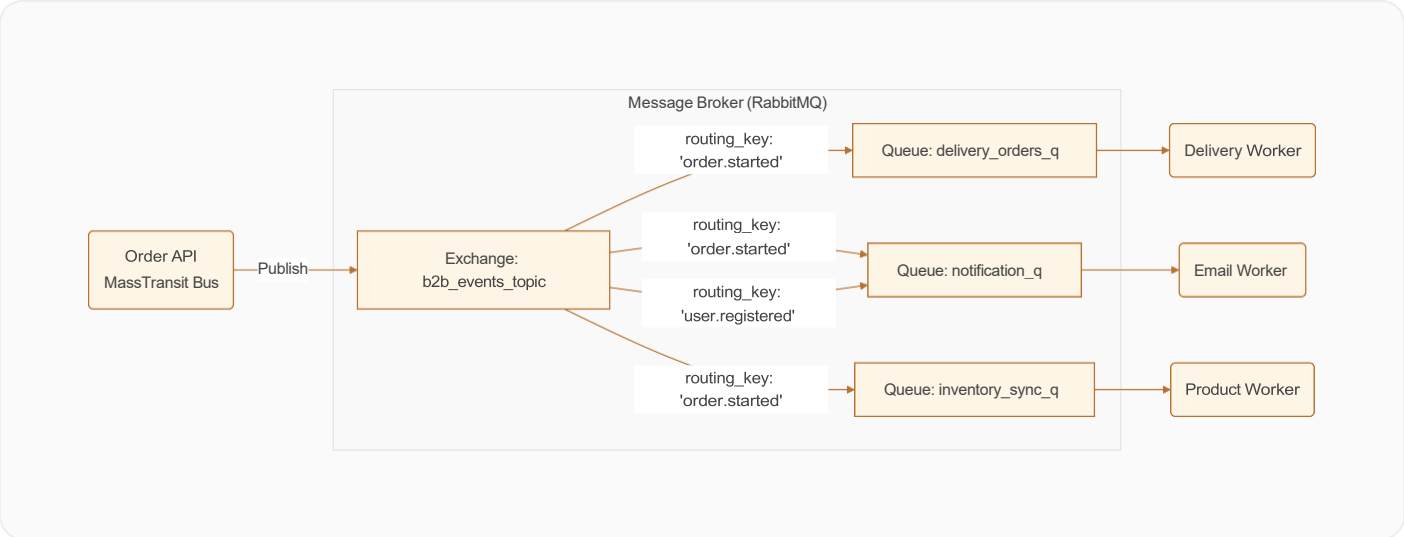

Order Confirmed Event Received
Deserialize Event Payload
Iterate Order Items
Lookup Stock in Product DB
Sufficient
Deduct Quantity
Insufficient
Is New Stock < ReorderLevel?
Yes
No
Publish 'LowStock' Event
Raise 'Shortage' Alert Protocol
Save DbContext Log Trace

Activity Semantics

The inventory subsystem is completely detached from the checkout process (via the asynchronous event consumption depicted). The system employs an "Optimistic Sell" strategy: the user's cart reserves visually, but physical hard-allocation only occurs downstream. In edge cases of concurrency (Admin drops stock manually at equivalent time of checkout), the `AlertAdmin` node prompts human intervention rather than failing the core payment cycle.

9. SERVICE FLOW DIAGRAM

The topology of the RabbitMQ implementation utilizing MassTransit abstract layers. Differentiating between direct routing and topic broadcast paradigms within the fabric.



Message Contract (C# Integration Event)

```
public record OrderConfirmedIntegrationEvent
{
    public Guid CorrelationId { get; init; } = Guid.NewGuid();
    public Guid OrderId { get; init; }
    public Guid ClientId { get; init; }
    public string ClientEmail { get; init; }
    public decimal Amount { get; init; }
    public IEnumerable Items { get; init; }
    public DateTime OccurredOn { get; init; } = DateTime.UtcNow;
}
```

Idempotency Guarantees: Consumers monitor the `CorrelationId` or `OrderId` to ensure duplicate message delivery does not result in duplicate inventory deductions.

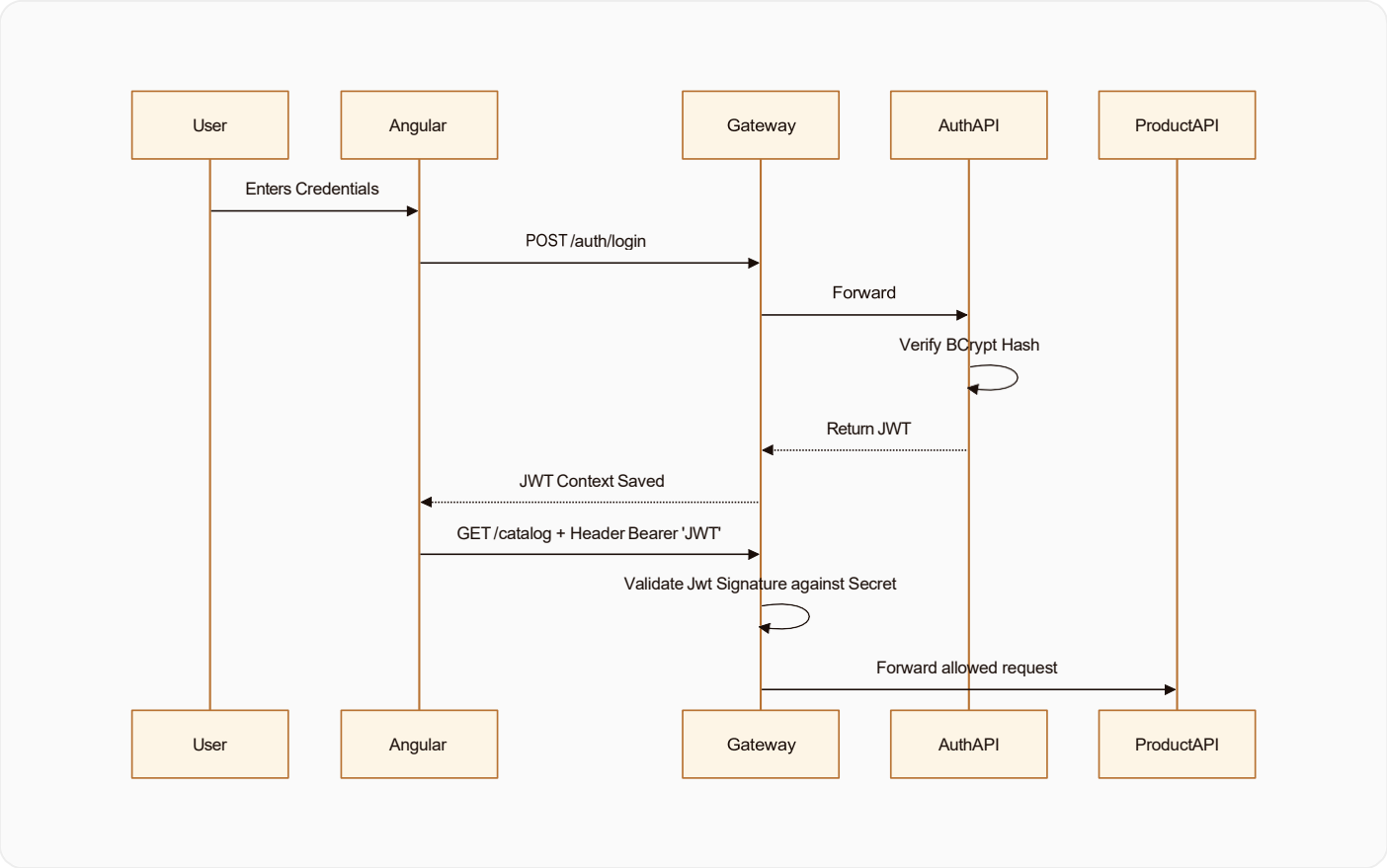
Security Fabric & Access Gontrol

Ember & Bean operates on a zero-trust model internally; JWT tokens must accompany all requests bridging the Ocelot Gateway.

Authentication &Authorization Model

Token Structure: Bearer tokens generated via ASP.NET Core Identity utilizing HS256 encryption. Payload explicitly includes customized claims:

- `sub` : UserId
- `role` : "Admin" or "Client"
- `isApproved` : Boolean indicating if SuperAdmin vetted the account (Crucial for B2B portal unlocking).



Access control to API endpoints applies declarative attributes on controllers: `[Authorize(Roles = "Admin")]` . Ocelot is configured to forward Claims transparently down into downstream services.

10. API Contract & Payload Definitions

Explicit mapping of vital endpoints facilitating the core order application flow.

1. Create Registration (Auth Service)

```
POST /api/auth/register

 Request Payload:
{
  "email": "corp@partner.com",
  "password": "SecureString#1",
  "companyName": "Partner Cafe Ltd."
}
Response: 201 Created (Approval pending)
```

2. Fetch Catalog (Product Service)

```
GET /api/products?page=1&limit=50

 Response:
{
  "data": [
    {
      "id": "guid",
      "sku": "ETH-YRG-01",
      "name": "Ethiopian Yirgacheffe",
      "availableStock": 250,
      "price": 18.50
    }
  ]
}
```

3. Request Verification Signature (Order Service)

```
POST /api/orders/verify

 Request Payload:
{
  "orderId": "internal-guid",
  "razorpayOrderId": "order_XXXXXX",
  "razorpayPaymentId": "pay_XXXXXXX",
```

```
  "razorpaySignature": "computed_hmac_hash"
}
Response: 200 OK
{
  "status": "PaymentVerified",
  "receiptUri": "/orders/receipt/internal-guid"
}
```

11. Resiliency & Error Handling Strategies

System stability metrics define behavior under adverse edge-case networking scenarios.

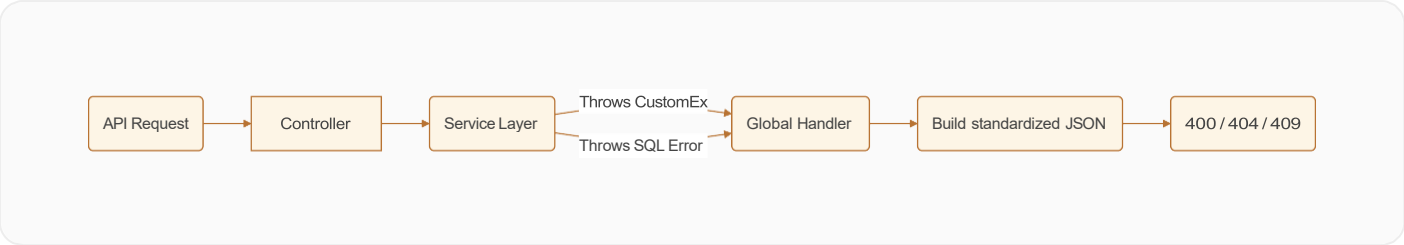
Global Exception Handling Middlewares

Every ASP.NET Core WebAPI injects a centralized `GlobalExceptionHandlerMiddleware`. Native generic errors resulting in 500s are masked to prevent leakages of Stacktraces. A standard `ProblemDetails` wrapper informs frontend consumers uniformly.

Database Concurrency Resolution

When multiple Admin dashboard users attempt to update an Order Status symmetrically:

1. Entity Framework throws `DbUpdateConcurrencyException`.
2. The API catches this and returns a `409 Conflict`.
3. Angular identifies the 409 and prompts: *"Record modified by another user. Reloading latest data."*



Retry Policies (Polly Integration)

HTTP outbound requests (e.g., Calling Razorpay servers or Groq AI) wrap inside Polly retry logic. If the Groq endpoint timeout executes, a transient retry policy allows up to 3 geometric backoffattempts before classifying the Chatbot microservice as temporarily degraded.